DESIGN WALKTHROUGH · OCTOBER 2026

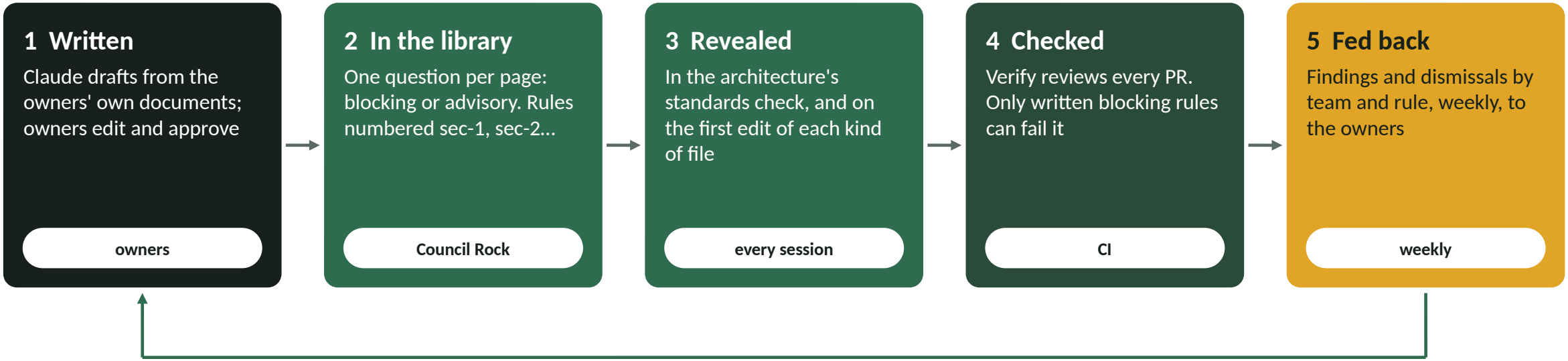
Standards and review

Written once by their owners, applied everywhere

How standards are written, reach Claude, are checked on every PR, and how dangerous actions are blocked

sec-1
sec-2
sec-3

A standard's life at a glance



Owners fix rules that are unclear or wrong

Separately, blocked outright for Claude

- Deploy commands
- Force-push
- Production credentials
- Edits to protected paths

A standard page

```yaml
---
type: standard
id: standards/org/secrets
owners: ["@seeonee/security"]
applies_to: []     # everywhere
enforcement: blocking
prefix: sec
---
# Secrets and personal data
- sec-1 Never commit a secret; use the vault.
        Why: git history is copied everywhere.
- sec-2 Retired: replaced by sec-4.
- sec-3 Never log a card number.
        Example: log the last four digits.
```

One question per page

Blocking or advisory, as a whole. A topic with both becomes two pages.

Numbered rules

sec-1, sec-2… never reused. A dropped rule stays, marked retired. Lower case, so never mistaken for a Jira key.

Owned by a team

The security team owns security standards. GitHub's owners file follows each page's owners.

Writing the first standards

1. **Start**
   An owner runs /bagheera:draft-standard with their existing documents

2. **Draft**
   Claude writes the page: numbered rules, a reason and an example each

3. **Report**
   What it left out: linter rules, vague advice, duplicates, conflicts

4. **PR**
   A library PR to the page's owners; the usual checks run

5. **Approve**
   Owners edit and approve. They vouch for every line

What comes first: only what the pilot needs

Secrets and personal data	Security team
Deployment and environments	Platform or ops team
The pilot team's languages	Pilot tech leads

Two rules for every standard

- Every rule can be checked from a diff; vague advice is left out
- Never restate what a linter enforces; link to its settings

Standards reach Claude when the work needs them

Architecture
The blocking standards for the services involved, in the standards check

First edit
That kind of file's standards, the service's and the org-wide ones, once each

Verify
The standards that apply to the files in the PR's diff

On demand
Any standard, through the knowledge skill

The first-edit hook, in every session

Claude edits a file
In any session, stage command or not

→

The hook reads its kind
.go, .ts, a Dockerfile, a Helm chart…

→

It adds the standards
Blocking first; about 2,000 tokens at most

→

Claude re-checks the edit
And carries on with the standards in view

Works in subagents too. A session that never edits code loads no standards.

The standards check in the architecture

```
## Standards check

sec-1    Reads the payment key from the vault
sec-3    Refund logs carry the last four digits
deploy-2 Exception: the refund job runs from
         stage for one release, while ops
         moves it. Tech lead to confirm.
```

- Blocking rules only, and only those that bear on the design
- Filled from the current standards on every run; no separate principles file
- Replaces Spec Kit's constitution (PRD R5)

What happens to an exception

1. **Written in the check**
   With its reason, in architecture.md
2. **Approved by the tech lead**
   /approved architecture covers it
3. **Still flagged by Verify**
   The code breaks a blocking rule, so the finding appears
4. **Dismissed, citing the architecture**
   By the tech lead, on record
5. **Seen by the rule's owners**
   In the weekly report

The PR review: what can fail it

Standards The change against the standards that apply	fails	*blocking rules*
Security General review beyond the written rules	advises	*unless a rule covers it*
Matches the plan Ticket branches: does what the phase says	advises	
Feature folder Ticket branches: only its own ticket's files	fails	*a script*
Criteria coverage Phase PRs: every criterion has a test	fails	*a script*
Tests first Phase PRs: the first commit is the tests	warns	*recorded*

A finding, on the line it's about

`sec-3 · blocking`

Never log a card number. This line logs card.number in full. Log the last four digits instead.

- A separate Claude; it never sees the author's session
- One summary comment; re-runs update, never repeat
- Every PR in onboarded repos, Bagheera or not

When the reviewer is wrong, and what owners learn

Who	A tech lead of the repo's domain, never the PR's author
How	Reply on the finding: `/dismiss sec-3 <reason>`
Then	Verify runs again without it: on the next push, or within 15 minutes
Scope	That finding on that PR only

Every dismissal is recorded on the PR's check.

Weekly, by team and rule (illustrative)

Rule	Findings	Dismissed	Mostly in
`sec-3`	12	1	payments
`go-7`	9	6	search
`deploy-2`	4	0	payments

Many dismissals

The rule is probably unclear or wrong: its owners fix it (go-7)

One team, often

Something that team hasn't understood: for managers

Forbidden actions: what truly blocks them

	Deny rules · everyone	GitHub's rules	Sandbox · pilot only
Force-push	usual forms	**truly blocked**	—
Protected paths	usual forms	**truly blocked**	repo only
Deploy commands	usual forms	—	**truly blocked**
Production credentials	usual forms	—	**truly blocked**

"The usual forms": a deny rule matches the command as Claude usually writes it, not the same program by full path, inside sh -c or from a script.

Bypass mode off
for everyone, in managed settings

Red-team first
before the pilot, both setups

Sandbox for all?
decided at pilot exit

Questions to refine together

1. **First owners**
   Who owns secrets, deployment and the pilot's languages, by name?

2. **Prefixes**
   Who hands them out, and should domain standards carry the domain?

3. **Who may dismiss**
   Domain tech leads only, or also the rule's owners?

4. **Security with no rule**
   Advice only, or should some kinds always block?

5. **Deny list**
   What are our deploy tools and protected paths, exactly?

6. **Sandbox trial**
   Which hosts must the pilot reach? Anyone on native Windows?

7. **Reveal limit**
   Is about 2,000 tokens enough for a large service's standards?

8. **Re-run wait**
   Is up to 15 minutes after a dismissal acceptable?

Next in the design: status and the dashboard (design doc 5).